Principle 2. ***Travel light—don't create more models than you need.*** Every model that is created must be kept up to date as changes occur. More importantly, every new model takes time that might otherwise be spent on construction (coding and testing). Therefore, create only those models that make it easier and faster to construct the software.

Principle 3. ***Strive to produce the simplest model that will describe the problem or the software.*** Don't overbuild the software [Amb02]. By keeping models simple, the resultant software will also be simple. The result is software that is easier to integrate, easier to test, and easier to maintain (to change). In addition, simple models are easier for members of the software team to understand and critique, resulting in an ongoing form of feedback that optimizes the end result.

Principle 4. ***Build models in a way that makes them amenable to change.*** Assume that your models will change, but in making this assumption don't get sloppy. The problem with this attitude is that you may not create a reasonably complete requirements model, which means you'll create a design (design model) that will invariably miss important functions and features.

Principle 5. ***Be able to state an explicit purpose for each model that is created.*** Every time you create a model, ask yourself why you're doing so. If you can't provide solid justification for the existence of the model, don't spend time on it.

Principle 6. ***Adapt the models you develop to the system at hand.*** It may be necessary to adapt model notation or rules to the application; for example, a video game application might require a different modeling technique than real-time, embedded software for adaptive cruise control in an automobile.

Principle 7. ***Try to build useful models, but forget about building perfect models.*** When building requirements and design models, a software engineer reaches a point of diminishing returns. That is, the effort required to make a model that is complete and internally consistent may not be worth the benefits of these properties. Iterating endlessly to make a model "perfect" does not serve the need for agility.

Principle 8. ***Don't become dogmatic about the syntax of the model. If it communicates content successfully, representation is secondary.*** Although everyone on a software team should try to use consistent notation during modeling, the most important characteristic of the model is to communicate information that enables the next software engineering task. If a model does this successfully, incorrect syntax can be forgiven.

Principle 9. ***If your instincts tell you a model isn't right even though it seems okay on paper, you probably have reason to be concerned.*** If you are an experienced software engineer, trust your instincts. Software work teaches many lessons—some of them on a subconscious level. If something tells you that a design model is doomed to fail (even though you can't prove it explicitly), you have reason to spend additional time examining the model or developing a different one.

Principle 10. ***Get feedback as soon as you can.*** The intent of any model is to communicate information. It should stand on its own. Assume that you won't be there to explain the model. Every model should be reviewed by members of the software team. The intent of these reviews is to provide feedback that can be used to correct modeling mistakes, change misinterpretations, and add features or functions that were inadvertently omitted.

6.2.4 Construction Principles

The construction activity encompasses a set of coding and testing tasks that lead to operational software that is ready for delivery to the customer or end user.

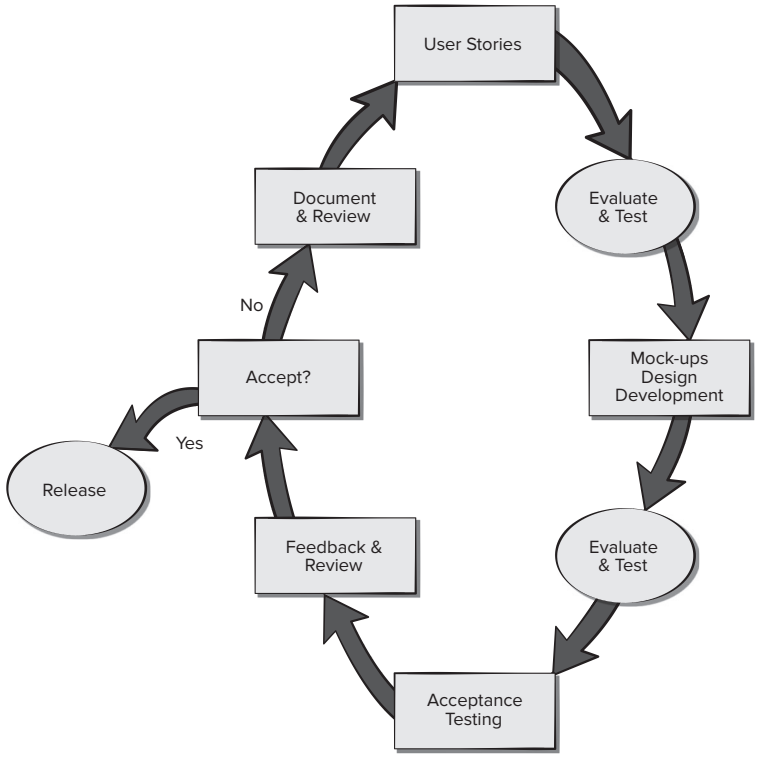
In modern software engineering work, coding may be: (1) the direct creation of programming language source code, (2) the automatic generation of source code using an intermediate design like representation of the component to be built, or (3) the automatic generation of executable code using a fourth-generation programming language (e.g., Unreal4 *Blueprints*).[6]

The initial focus of testing is at the component level, often called *unit testing*. Other levels of testing include (1) *integration testing* (conducted as the system is constructed), (2) *validation testing* that assesses whether requirements have been met for the complete system (or software increment), and (3) *acceptance testing* that is conducted by the customer in an effort to exercise all required features and functions. Figure 6.5 shows where testing and test case design is placed in agile processes.

Testing is considered in detail in Chapters 19 through 21. The following set of fundamental principles and concepts are applicable to coding and testing.

FIGURE 6.5

Testing in agile processes



6 *Blueprints* is a visual scripting tool created by Epic Games (https://docs.unrealengine.com/latest/INT/Engine/Blueprints/).

Coding Principles. The principles that guide the coding task are closely aligned with programming style, programming languages, and programming methods. There are some fundamental principles that might be followed:

Preparation Principles: Before you write one line of code, be sure you:

Principle 1. ***Understand the problem you're trying to solve.***

Principle 2. ***Understand basic design principles and concepts.***

Principle 3. ***Pick a programming language that meets the needs of the software to be built and the environment in which it will operate.***

Principle 4. ***Select a programming environment that provides tools that will make your work easier.***

Principle 5. ***Create a set of unit tests that will be applied once the component you code is completed.***

Coding Principles: As you begin writing code, be sure you:

Principle 6. ***Constrain your algorithms by following structured programming [Boh00] practice.***

Principle 7. ***Consider the use of pair programming.***

Principle 8. ***Select data structures that will meet the needs of the design.***

Principle 9. ***Understand the software architecture and create interfaces that are consistent with it.***

Validation Principles: After you've completed your first coding pass, be sure you:

Principle 10. ***Conduct a code walkthrough when appropriate.***

Principle 11. ***Perform unit tests and correct errors you've uncovered.***

Principle 12. ***Refactor the code to improve its quality.***

Testing Principles. In a classic book on software testing, Glen Myers [Mye79] states a number of rules that can serve well as testing objectives:

1. Testing is a process of executing a program with the intent of finding an error.
2. A good test case is one that has a high probability of finding an as-yet-undiscovered error.
3. A successful test is one that uncovers an as-yet-undiscovered error.

These objectives imply a dramatic change in viewpoint for some software developers. They move counter to the commonly held view that a successful test is one in which no errors are found. Your objective is to design tests that systematically uncover different classes of errors and to do so with a minimum amount of time and effort.

As a secondary benefit, testing demonstrates that software functions appear to be working according to specification and that behavioral and performance requirements appear to have been met. In addition, the data collected as testing is conducted provide a good indication of software reliability and some indication of software quality.

FIGURE 6.6

Testing is never complete



Testing cannot show the absence of errors and defects; it can show only that software errors and defects are present (Figure 6.6). It is important to keep this (rather gloomy) statement in mind as testing is being conducted.

Davis [Dav95b] suggests a set of testing principles[7] that have been adapted for use in this book. In addition, Everett and Meyer [Eve09] suggest additional principles:

Principle 1. ***All tests should be traceable to customer requirements.***[8] The objective of software testing is to uncover errors. It follows that the most severe defects (from the customer's point of view) are those that cause the program to fail to meet its requirements.

Principle 2. ***Tests should be planned long before testing begins.*** Test planning (Chapter 19) can begin as soon as the requirements model is complete. Detailed definition of test cases can begin as soon as the design model has been solidified. Therefore, all tests can be planned and designed before any code has been generated.

Principle 3. ***The Pareto principle applies to software testing.*** In this context, the Pareto principle implies that 80 percent of all errors uncovered during testing will

7 Only a small subset of Davis's testing principles are noted here. For more information, see [Dav95b].

8 This principle refers to *functional tests*, that is, tests that focus on requirements. *Structural tests* (tests that focus on architectural or logical detail) may not address specific requirements directly.

likely be traceable to 20 percent of all program components. The problem, of course, is to isolate these suspect components and to thoroughly test them.

Principle 4. ***Testing should begin "in the small" and progress toward testing "in the large."*** The first tests planned and executed generally focus on individual components. As testing progresses, focus shifts to looking for errors in integrated clusters of components and ultimately in the entire system.

Principle 5. ***Exhaustive testing is not possible.*** The number of path permutations for even a moderately sized program is exceptionally large. For this reason, it is impossible to execute every combination of paths during testing. It is possible, however, to adequately cover program logic and to ensure that all conditions in the component-level design have been exercised.

Principle 6. ***Apply to each module in the system a testing effort commensurate with its expected fault density.*** These are often the newest modules or the ones that are least understood by the developers.

Principle 7. ***Static testing techniques can yield high results.*** More than 85 percent of software defects originate in the software documentation (requirements, specifications, code walk-throughs, and user manuals) [Jon91]. There may be value in testing the system documentation.

Principle 8. ***Track defects and look for patterns in defects uncovered by testing.*** The total defects uncovered is a good indicator of software quality. The types of defects uncovered can be a good measure of software stability. Patterns of defects found over time can forecast numbers of expected defects.

Principle 9. ***Include test cases that demonstrate software is behaving correctly.*** As software components are being maintained or adapted, unexpected interactions cause unintended side effects in other components. It is important to have a set of regression test cases (Chapter 19) ready to check system behavior after changes are made to a software product.

6.2.5 Deployment Principles

As we noted in Part One of this book, the deployment activity encompasses three actions: delivery, support, and feedback. Because modern software process models are evolutionary or incremental in nature, deployment happens not once, but several times as software moves toward completion. Each delivery cycle provides the customer and end users with an operational software increment that provides usable functions and features. Each support cycle provides documentation and human assistance for all functions and features introduced during all deployment cycles to date. Each feedback cycle provides the software team with important guidance that results in modifications to the functions, features, and approach taken for the next increment. Typical deployment actions are illustrated in Figure 6.7.

The delivery of a software increment represents an important milestone for any software project. Some key principles should be followed as the team prepares to deliver an increment:

Principle 1. ***Customer expectations for the software must be managed.*** Too often, the customer expects more than the team has promised to deliver, and disappointment

FIGURE 6.7

Deployment actions



occurs immediately. This results in feedback that is not productive and ruins team morale. In her book on managing expectations, Naomi Karten [Kar94] states: "The starting point for managing expectations is to become more conscientious about what you communicate and how." She suggests that a software engineer must be careful about sending the customer conflicting messages (e.g., promising more than you can reasonably deliver in the time period provided or delivering more than you promise for one software increment and then less than promised for the next).

Principle 2. ***A complete delivery package should be assembled and tested.*** All executable software, support data files, support documents, and other relevant information should be assembled and thoroughly beta-tested with actual users. All installation scripts and other operational features should be thoroughly exercised in all possible computing configurations (i.e., hardware, operating systems, peripheral devices, networking arrangements).

Principle 3. ***A support regimen must be established before the software is delivered.*** An end user expects responsiveness and accurate information when a question or problem arises. If support is ad hoc, or worse, nonexistent, the customer will become dissatisfied immediately. Support should be planned, support materials should be prepared, and appropriate record-keeping mechanisms should be established so that the software team can conduct a categorical assessment of the kinds of support requested.

Principle 4. ***Appropriate instructional materials must be provided to end users.*** The software team delivers more than the software itself. Appropriate training aids (if required) should be developed; troubleshooting guidelines should be provided,